

Algorithm Design IX

Advanced Data Structures

Guoqiang Li
School of Computer Science

Shanghai Jiao Tong University

Review of Previous Lecture

Lengths on Edges

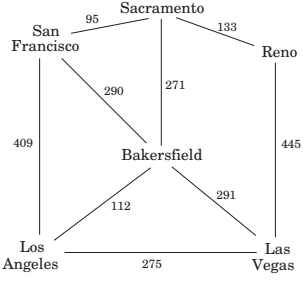
BFS treats all edges as having the same length.

It is rarely true in applications where shortest paths are to be found.

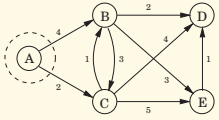
Every edge $e \in E$ with a length l_e.

If $e = (u, v)$, we will sometimes also write

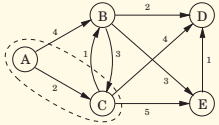
$$l(u, v) \qquad \text{or} \qquad l_{uv}$$



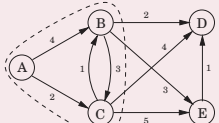
An Example



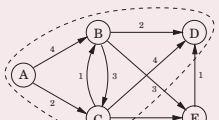
A: 0	D: ∞
B: 4	E: ∞
C: 2	



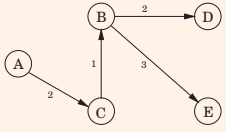
A: 0	D: 6
B: 3	E: 7
C: 2	



A: 0	D: 5
B: 3	E: 6
C: 2	



A: 0	D: 5
B: 3	E: 6
C: 2	



Dijkstra's Shortest-Path Algorithm

DIJKSTRA(G, l, s)
input : Graph $G = (V, E)$, directed or undirected; positive edge length $\{l_e \mid e \in E\}$; Vertex $s \in V$
output: For all vertices u reachable from s, $dist(u)$ is the set to the distance from s to u

for *all* $u \in V$ **do**
  $dist(u) = \infty$; $prev(u) = nil$;
end
$dist(s) = 0$;
H =makequeue (V) \\ *using dist-values as keys*;
while H *is not empty* **do**
  u=deletemin (H);
  for *all edge* $(u, v) \in E$ **do**
    if $dist(v) > dist(u) + l(u, v)$ **then**
      $dist(v) = dist(u) + l(u, v)$; $prev(v) = u$;
      decreasekey (H,v);
    end
  end
end

Running Time

Since `makequeue` takes at most as long as $|V|$ insert operations, we get a total of $|V|$ `deletemin` and $|V| + |E|$ `insert/decreasekey` operations.

Priority Queue

Priority Queue

Priority queue is a data structure usually implemented by heap.

Priority Queue

Priority queue is a data structure usually implemented by heap.

- Insert: Add a new element to the set.

Priority Queue

Priority queue is a data structure usually implemented by heap.

- Insert: Add a new element to the set.
- Decrease-key: Accommodate the decrease in key value of a particular element.

Priority Queue

Priority queue is a data structure usually implemented by heap.

- Insert: Add a new element to the set.
- Decrease-key: Accommodate the decrease in key value of a particular element.
- Delete-min: Return the element with the smallest key, and remove it from the set.

Priority Queue

Priority queue is a data structure usually implemented by heap.

- Insert: Add a new element to the set.
- Decrease-key: Accommodate the decrease in key value of a particular element.
- Delete-min: Return the element with the smallest key, and remove it from the set.
- Make-queue: Build a priority queue out of the given elements, with the given key values. (In many implementations, this is significantly faster than inserting the elements one by one.)

The simplest implementation is as an unordered array of key values for all potential elements.

Array

The simplest implementation is as an unordered array of key values for all potential elements.

An insert or `decreasekey` is fast, because it just involves adjusting a key value, an $O(1)$ operation.

The simplest implementation is as an unordered array of key values for all potential elements.

An insert or `decreasekey` is fast, because it just involves adjusting a key value, an $O(1)$ operation.

To `deletemin`, on the other hand, requires a linear-time scan of the list.

Elements are stored in a complete binary tree.

Elements are stored in a complete binary tree.

In addition, The key value of any node of the tree is less than or equal to that of its children.

Elements are stored in a complete binary tree.

In addition, The key value of any node of the tree is less than or equal to that of its children.

In particular, therefore, the root always contains the smallest element.

The regularity of a complete binary tree makes it easy to represent using an array.

The regularity of a complete binary tree makes it easy to represent using an array.

The tree nodes have a natural ordering: row by row, starting at the root and moving left to right within each row.

The regularity of a complete binary tree makes it easy to represent using an array.

The tree nodes have a natural ordering: row by row, starting at the root and moving left to right within each row.

If there are n nodes, this ordering specifies their positions $1, 2, \ldots, n$ within the array.

The regularity of a complete binary tree makes it easy to represent using an array.

The tree nodes have a natural ordering: row by row, starting at the root and moving left to right within each row.

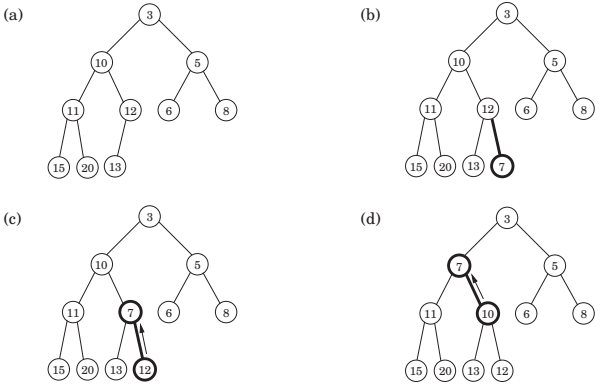
If there are n nodes, this ordering specifies their positions $1, 2, \ldots, n$ within the array.

Moving up and down the tree is easily simulated on the array, using the fact that node number j has parent $\lfloor j/2 \rfloor$ and children $2j$ and $2j + 1$.

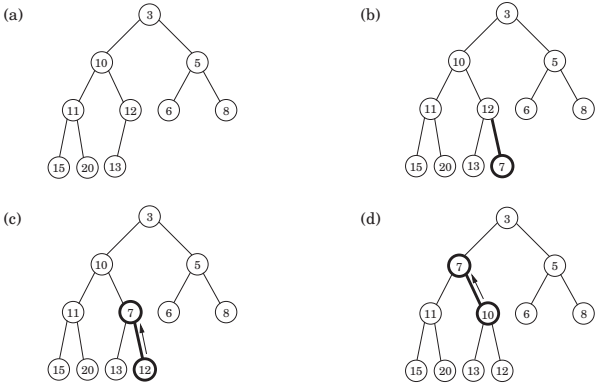
To insert, place the new element at the bottom of the tree (in the first available position), and let it "bubble up".

To insert, place the new element at the bottom of the tree (in the first available position), and let it "bubble up".

The number of swaps is at most the height of the tree $\lfloor \log_2 n \rfloor$, when there are n elements.



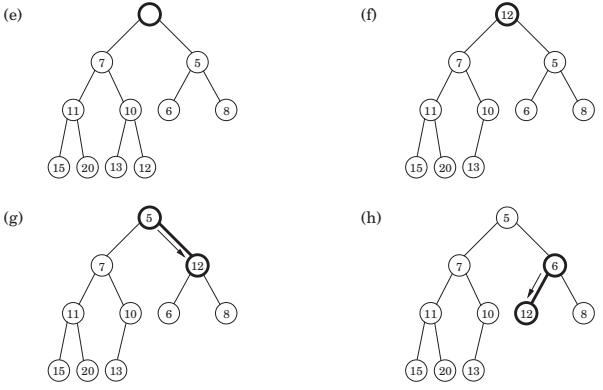
A `decreasekey` is similar, except the element is already in the tree, so we let it bubble up from its current position.



To `deletemin`, return the root value, and remove it from the heap, take the last node in the tree (in the rightmost position in the bottom row) and place it at the root.

To `deletemin`, return the root value, and remove it from the heap, take the last node in the tree (in the rightmost position in the bottom row) and place it at the root.

Then let it "shift down". Again this takes $O(\log n)$ time.



d-ary heap

A d-ary heap is identical to a binary heap, except that nodes have d children.

A d-ary heap is identical to a binary heap, except that nodes have d children.

The array representation of the d-ary:

d-ary heap

A d-ary heap is identical to a binary heap, except that nodes have d children.

The array representation of the d-ary: the node number j has parent $\lceil (j-1)/d \rceil$

d-ary heap

A d-ary heap is identical to a binary heap, except that nodes have d children.

The array representation of the d-ary: the node number j has parent $\lceil (j-1)/d \rceil$and children $\{(j-1)d+2, \ldots, \min\{n, (j-1)d+d+1\}\}$

This reduces the height of a tree with n elements to

$$\Theta(\log_d n) = \Theta((\log n)/(\log d))$$

d-ary heap

A d-ary heap is identical to a binary heap, except that nodes have d children.

The array representation of the d-ary: the node number j has parent $\lceil (j-1)/d \rceil$and children $\{(j-1)d+2, \ldots, \min\{n, (j-1)d+d+1\}\}$

This reduces the height of a tree with n elements to

$$\Theta(\log_d n) = \Theta((\log n)/(\log d))$$

`Inserts` are therefore speeded up by a factor of $\Theta(\log d)$.

d-ary heap

A d-ary heap is identical to a binary heap, except that nodes have d children.

The array representation of the d-ary: the node number j has parent $\lceil (j-1)/d \rceil$and children $\{(j-1)d+2, \ldots, \min\{n, (j-1)d+d+1\}\}$

This reduces the height of a tree with n elements to

$$\Theta(\log_d n) = \Theta((\log n)/(\log d))$$

`Inserts` are therefore speeded up by a factor of $\Theta(\log d)$.

`Deletemin` operations, however, take a little longer, namely $O(d \log_d n)$.

A Fibonacci heap is a collection of min-heap-ordered trees. Trees within Fibonacci heaps are rooted but unordered linked by a circular, doubly linked list.

A Fibonacci heap is a collection of min-heap-ordered trees. Trees within Fibonacci heaps are rooted but unordered linked by a circular, doubly linked list.

Each node x contains a pointer $p[x]$ to its parent and a pointer $child[x]$ to any one of its children.

A Fibonacci heap is a collection of min-heap-ordered trees. Trees within Fibonacci heaps are rooted but unordered linked by a circular, doubly linked list.

Each node x contains a pointer $p[x]$ to its parent and a pointer $child[x]$ to any one of its children.

The children of x are linked together in a circular, doubly linked list, which we call the child list of x.

A Fibonacci heap is a collection of min-heap-ordered trees. Trees within Fibonacci heaps are rooted but unordered linked by a circular, doubly linked list.

Each node x contains a pointer $p[x]$ to its parent and a pointer $child[x]$ to any one of its children.

The children of x are linked together in a circular, doubly linked list, which we call the child list of x.

Each child y in a child list has pointers $left[y]$ and $right[y]$ that point to y's left and right siblings, respectively.

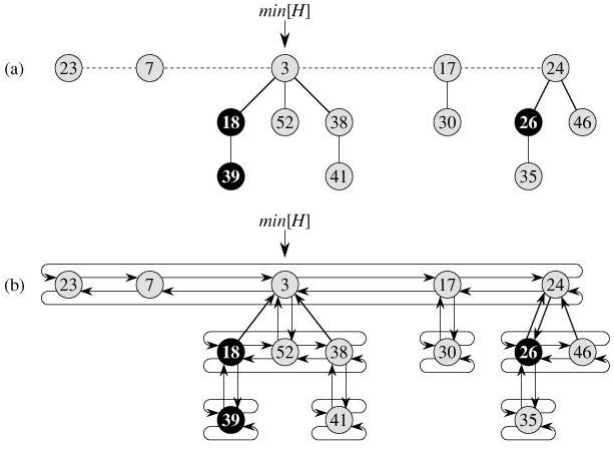
A Fibonacci heap is a collection of min-heap-ordered trees. Trees within Fibonacci heaps are rooted but unordered linked by a circular, doubly linked list.

Each node x contains a pointer $p[x]$ to its parent and a pointer $child[x]$ to any one of its children.

The children of x are linked together in a circular, doubly linked list, which we call the child list of x.

Each child y in a child list has pointers $left[y]$ and $right[y]$ that point to y's left and right siblings, respectively.

If node y is an only child, then $left[y] = right[y] = y$. The order in which siblings appear in a child list is arbitrary.


min[H]
(a)
23
7
3
17
24
18
52
38
30
26
46
39
41
35
min[H]
(b)
23
7
3
17
24
18
52
38
30
26
46
39
41
35

Which Heap is Best

Implementation	deletemin	insert/decreasekey	$\lvert V\rvert \times$ deletemin$+(\lvert V\rvert+\lvert E\rvert)\times$ insert
Array	$O(\lvert V\rvert)$	$O(1)$	$O(\lvert V\rvert^2)$
Binary heap	$O(\log \lvert V\rvert)$	$O(\log \lvert V\rvert)$	$O((\lvert V\rvert+\lvert E\rvert)\log \lvert V\rvert)$
d-ary heap	$O(\frac{d \log \lvert V\rvert}{\log d})$	$O(\frac{\log \lvert V\rvert}{\log d})$	$O(\frac{(d\lvert V\rvert+\lvert E\rvert)\log \lvert V\rvert}{\log d})$
Fibonacci heap	$O(\log \lvert V\rvert)$	$O(1)$ (amortized)	$O(\lvert V\rvert \log \lvert V\rvert+\lvert E\rvert)$

Which heap is Best

A naive array implementation gives a respectable time complexity of $O(|V|^2)$, whereas with a binary heap we get $O((|V| + |E|) \log |V|)$. Which is preferable?

Which heap is Best

A naive array implementation gives a respectable time complexity of $O(|V|^2)$, whereas with a binary heap we get $O((|V| + |E|) \log |V|)$. Which is preferable?

This depends on whether the graph is sparse or dense.

Which heap is Best

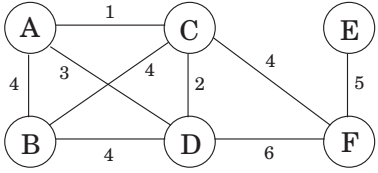
A naive array implementation gives a respectable time complexity of $O(|V|^2)$, whereas with a binary heap we get $O((|V| + |E|) \log |V|)$. Which is preferable?

This depends on whether the graph is sparse or dense.

- $|E|$ is less than $|V|^2$. If it is $\Omega(|V|^2)$, then clearly the array implementation is the faster.
- On the other hand, the binary heap becomes preferable as soon as $|E|$ dips below $|V|^2 / \log |V|$.
- The d-ary heap is a generalization of the binary heap and leads to a running time that is a function of d. The optimal choice is $d \approx |E|/|V|$;

Review of Previous Lecture

Build a Network

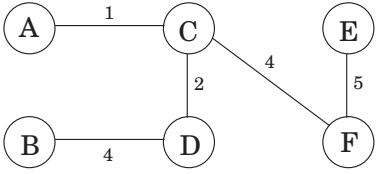


Suppose you are asked to network a collection of computers by linking selected pairs of them.

This translates into a graph problem in which

- nodes are computers,
- undirected edges are potential links, each with a maintenance cost.

Build a Network



The goal is to

- pick enough of these edges that the nodes are connected,
- the total maintenance cost is minimum.

One immediate observation is that the optimal set of edges cannot contain a cycle.

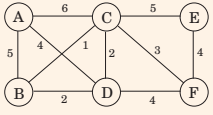
Kruskal's minimum spanning tree algorithm starts with the empty graph and then selects edges from E according to the following rule.

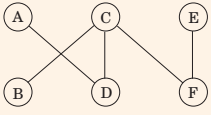
Repeatedly add the next lightest edge that doesn't produce a cycle.

Example

Starting with an empty graph and then attempt to add edges in increasing order of weight

$$B-C; C-D; B-D; C-F; D-F; E-F; A-D; A-B; C-E; A-C$$





The Kruskal's Algorithm

KRUSKAL(G, w)
input : A connected undirected graph $G = (V, E)$, with edge weight w_e
output: A minimum spanning tree defined by the edges X

for *all* $u \in V$ **do**
  `makeset` (u);
end
$X = \{\}$;
Sort the edges E by weight;
for *all* $(u, v) \in E$ *in increasing order of weight* **do**
  if `find` *(u)*$\neq$`find` *(v)* **then**
    add (u, v) to X;
    `union` (u,v)
  end
end

Disjoint Sets

`makeset`(x)	create a singleton set containing x	$\|V\|$
`find`(x)	find the set that x belong to	$2 \cdot \|E\|$
`union`(x, y)	merge the sets containing x and y	$\|V\| - 1$

Disjoint Set

Union by rank

We store a set is by a directed tree.

Union by rank

We store a set is by a directed tree. Nodes of the tree are elements of the set, arranged in no particular order, and each has parent pointers that eventually lead up to the root of the tree.

Union by rank

We store a set is by a directed tree. Nodes of the tree are elements of the set, arranged in no particular order, and each has parent pointers that eventually lead up to the root of the tree.

This root element is a convenient representative, or name, for the set.

Union by rank

We store a set is by a directed tree. Nodes of the tree are elements of the set, arranged in no particular order, and each has parent pointers that eventually lead up to the root of the tree.

This root element is a convenient representative, or name, for the set. It is distinguished from the other elements by the fact that its parent pointer is a self-loop.

Union by rank

We store a set is by a directed tree. Nodes of the tree are elements of the set, arranged in no particular order, and each has parent pointers that eventually lead up to the root of the tree.

This root element is a convenient representative, or name, for the set. It is distinguished from the other elements by the fact that its parent pointer is a self-loop.

In addition to a parent pointer π, each node also has a rank that, for the time being, should be interpreted as the height of the subtree hanging from that node.

Union by Rank

MAKESET(x)

$\pi(x) = x$;
rank (x)=0;

Union by Rank

MAKESET(x)

$\pi(x) = x$;
rank (x)=0;

FIND(x)

while $x \neq \pi(x)$ **do**
  $x = \pi(x)$;
end
return(x);

Union by Rank


```
MAKESET(x)
π(x) = x;
rank (x)=0;
```

```
FIND(x)
while x ≠ π(x) do
    x = π(x);
end
return(x);
```

MAKESET is a constant-time operation.

Union by Rank


```
MAKESET(x)
π(x) = x;
rank (x)=0;
```

```
FIND(x)
while x ≠ π(x) do
    x = π(x);
end
return(x);
```

MAKESET is a constant-time operation.

FIND follows parent pointers to the root of the tree and therefore takes time proportional to the height of the tree.

Union by Rank


```
MAKESET(x)
π(x) = x;
rank (x)=0;
```

```
FIND(x)
while x ≠ π(x) do
    x = π(x);
end
return(x);
```

MAKESET is a constant-time operation.

FIND follows parent pointers to the root of the tree and therefore takes time proportional to the height of the tree.

The tree actually gets built via the third operation, UNION, and so we must make sure that this procedure keeps trees shallow.

Union

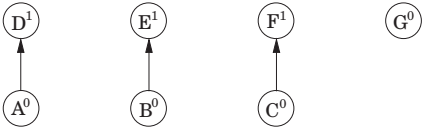

```
UNION(x, y)
r_x=find(x);
r_y=find(y);
if r_x = r_y then return;
if rank(r_x)>rank(r_y) then
    π(r_y) = r_x;
end
else
    π(r_x) = r_y;
    if rank(r_x)=rank(r_y) then rank(r_y)=rank(r_y)+1;
end
```

An Example

After $\texttt{makeset}(A), \texttt{makeset}(B), \ldots, \texttt{makeset}(G)$:

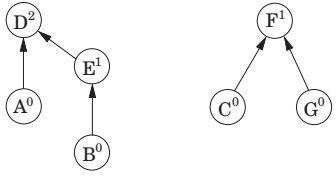


After $\texttt{union}(A, D), \texttt{union}(B, E), \texttt{union}(C, F)$:

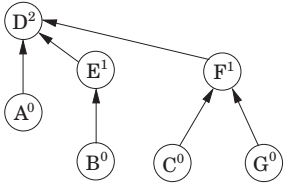


An Example

After $\texttt{union}(C, G), \texttt{union}(E, A)$:



After $\texttt{union}(B, G)$:



Lemma (1)

For any non-root x, $\texttt{rank}(x) < \texttt{rank}(\pi(x))$.

Lemma (1)

For any non-root x, $\texttt{rank}(x) < \texttt{rank}(\pi(x))$.

Proof Sketch:

Lemma (1)

For any non-root x, $\texttt{rank}(x) < \texttt{rank}(\pi(x))$.

Proof Sketch:

By design, the rank of a node is exactly the height of the subtree rooted at that node. This means, for instance, that as you move up a path toward a root node, the rank values along the way are strictly increasing.

Lemma (2)

Any root node of `rank` k has least 2^k nodes in its tree.

Lemma (2)

Any root node of `rank` k has least 2^k nodes in its tree.

Proof Sketch:

Lemma (2)

Any root node of `rank` k has least 2^k nodes in its tree.

Proof Sketch:

A root node with rank k is created by the merger of two trees with roots of rank $k-1$. By induction to get the results.

Lemma (3)

If there are n elements overall, there can be at most $n/2^k$ nodes of `rank` k.

Lemma (3)

If there are n elements overall, there can be at most $n/2^k$ nodes of `rank` k.

Proof Sketch:

Lemma (3)

If there are n elements overall, there can be at most $n/2^k$ nodes of `rank` k.

Proof Sketch:

A node of rank k has at least 2^k descendants.

Lemma (3)

If there are n elements overall, there can be at most $n/2^k$ nodes of `rank` k.

Proof Sketch:

A node of rank k has at least 2^k descendants.

Any internal node was once a root, and neither its rank nor its set of descendants has changed since then.

Lemma (3)

If there are n elements overall, there can be at most $n/2^k$ nodes of `rank` k.

Proof Sketch:

A node of rank k has at least 2^k descendants.

Any internal node was once a root, and neither its rank nor its set of descendants has changed since then.

Different rank-k nodes cannot have common descendants. Any element has at most one ancestor of rank k.

With the data structure as presented so far, the total time for Kruskal's algorithm becomes

With the data structure as presented so far, the total time for Kruskal's algorithm becomes

- $O(|E| \log |V|)$ for sorting the edges ($\log |V| \approx \log |E|$),

The Analysis

With the data structure as presented so far, the total time for Kruskal's algorithm becomes

- $O(|E| \log |V|)$ for sorting the edges ($\log |V| \approx \log |E|$),
- $O(|E| \log |V|)$ for the `union` and `find` operations that dominate the rest of the algorithm.

The Analysis

With the data structure as presented so far, the total time for Kruskal's algorithm becomes

- $O(|E| \log |V|)$ for sorting the edges ($\log |V| \approx \log |E|$),
- $O(|E| \log |V|)$ for the `union` and `find` operations that dominate the rest of the algorithm.

But what if the edges are given to us sorted? Or if the weights are small (say, $O(|E|)$) so that sorting can be done in linear time?

The Analysis

With the data structure as presented so far, the total time for Kruskal's algorithm becomes

- $O(|E| \log |V|)$ for sorting the edges ($\log |V| \approx \log |E|$),
- $O(|E| \log |V|)$ for the `union` and `find` operations that dominate the rest of the algorithm.

But what if the edges are given to us sorted? Or if the weights are small (say, $O(|E|)$) so that sorting can be done in linear time?

THEN THE DATA STRUCTURE PART BECOMES THE BOTTLENECK!

The Analysis

With the data structure as presented so far, the total time for Kruskal's algorithm becomes

- $O(|E| \log |V|)$ for sorting the edges ($\log |V| \approx \log |E|$),
- $O(|E| \log |V|)$ for the `union` and `find` operations that dominate the rest of the algorithm.

But what if the edges are given to us sorted? Or if the weights are small (say, $O(|E|)$) so that sorting can be done in linear time?

THEN THE DATA STRUCTURE PART BECOMES THE BOTTLENECK!

The main question:

The Analysis

With the data structure as presented so far, the total time for Kruskal's algorithm becomes

- $O(|E| \log |V|)$ for sorting the edges ($\log |V| \approx \log |E|$),
- $O(|E| \log |V|)$ for the `union` and `find` operations that dominate the rest of the algorithm.

But what if the edges are given to us sorted? Or if the weights are small (say, $O(|E|)$) so that sorting can be done in linear time?

THEN THE DATA STRUCTURE PART BECOMES THE BOTTLENECK!

The main question:

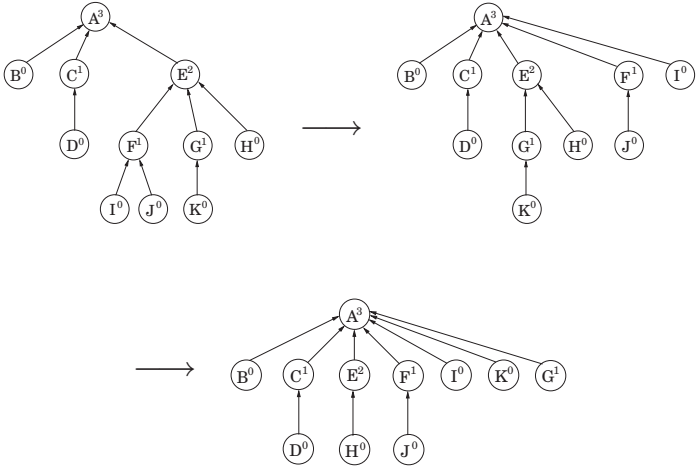
How can we perform `union`'s and `find`'s faster than $\log n$?

Path Compression

FIND(x)

if $x \neq \pi(x)$ **then** $\pi(x)$=FIND($\pi(x)$);
return($\pi(x)$);

Path Compression



$\texttt{FIND}(i) \to \texttt{FIND}(k)$

The benefit of this simple alteration is long-term rather than instantaneous and thus necessitates a particular kind of analysis.

The benefit of this simple alteration is long-term rather than instantaneous and thus necessitates a particular kind of analysis.

We need to look at sequences of `find` and `union` operations, starting from an empty data structure, and determine the average time per operation.

The benefit of this simple alteration is long-term rather than instantaneous and thus necessitates a particular kind of analysis.

We need to look at sequences of `find` and `union` operations, starting from an empty data structure, and determine the average time per operation.

This amortized cost turns out to be just barely more than $O(1)$, down from the earlier $O(\log n)$.

Think of the data structure as having a “top level” consisting of the root nodes,

Think of the data structure as having a “top level” consisting of the root nodes, and below it, the insides of the trees.

Think of the data structure as having a “top level” consisting of the root nodes, and below it, the insides of the trees.

There is a division of labor:

- `find` operations (with or without path compression) only touch the insides of trees,

Time Analysis

Think of the data structure as having a “top level” consisting of the root nodes, and below it, the insides of the trees.

There is a division of labor:

- `find` operations (with or without path compression) only touch the insides of trees,
- `union`’s only look at the top level.

Think of the data structure as having a “top level” consisting of the root nodes, and below it, the insides of the trees.

There is a division of labor:

- `find` operations (with or without path compression) only touch the insides of trees,
- `union`’s only look at the top level.

Thus path compression has no effect on `union` operations and leaves the top level unchanged.

We know that the ranks of root nodes are unaltered, but what about non-root nodes?

We know that the ranks of root nodes are unaltered, but what about non-root nodes?

The key point is that once a node ceases to be a root, it never resurfaces, and its rank is forever fixed.

Time Analysis

We know that the ranks of root nodes are unaltered, but what about non-root nodes?

The key point is that once a node ceases to be a root, it never resurfaces, and its rank is forever fixed.

Therefore the ranks of all nodes are unchanged by path compression, even though these numbers can no longer be interpreted as tree heights.

We know that the ranks of root nodes are unaltered, but what about non-root nodes?

The key point is that once a node ceases to be a root, it never resurfaces, and its rank is forever fixed.

Therefore the ranks of all nodes are unchanged by path compression, even though these numbers can no longer be interpreted as tree heights.

In particular,

We know that the ranks of root nodes are unaltered, but what about non-root nodes?

The key point is that once a node ceases to be a root, it never resurfaces, and its rank is forever fixed.

Therefore the ranks of all nodes are unchanged by path compression, even though these numbers can no longer be interpreted as tree heights.

In particular,

- For any x that is not a root, $rank(x) < rank(\pi(x))$

We know that the ranks of root nodes are unaltered, but what about non-root nodes?

The key point is that once a node ceases to be a root, it never resurfaces, and its rank is forever fixed.

Therefore the ranks of all nodes are unchanged by path compression, even though these numbers can no longer be interpreted as tree heights.

In particular,

- For any x that is not a root, $rank(x) < rank(\pi(x))$
- Any root node of `rank` k has least 2^k nodes in its tree.

We know that the ranks of root nodes are unaltered, but what about non-root nodes?

The key point is that once a node ceases to be a root, it never resurfaces, and its rank is forever fixed.

Therefore the ranks of all nodes are unchanged by path compression, even though these numbers can no longer be interpreted as tree heights.

In particular,

- For any x that is not a root, $rank(x) < rank(\pi(x))$
- Any root node of `rank` k has least 2^k nodes in its tree.
- If there are n elements overall, there can be at most $n/2^k$ nodes of `rank` k.

Time Analysis

If there are n elements, their rank values can range from 0 to $\log n$.

If there are n elements, their rank values can range from 0 to $\log n$.

Divide the nonzero part of this range into the following intervals:

$$\{1\}, \{2\}, \{3, 4\}, \{5, 6, \ldots, 16\}, \{17, 18, \ldots, 2^{16} = 65536\},$$
$$\{65537, 65538, ..., 2^{65536}\}, \ldots$$

If there are n elements, their rank values can range from 0 to $\log n$.

Divide the nonzero part of this range into the following intervals:

$$\{1\}, \{2\}, \{3, 4\}, \{5, 6, \ldots, 16\}, \{17, 18, \ldots, 2^{16} = 65536\},$$
$$\{65537, 65538, ..., 2^{65536}\}, \ldots$$

Each group is of the form $\{k + 1, k + 2, \ldots, 2^k\}$ where k is a power of 2.

If there are n elements, their rank values can range from 0 to $\log n$.

Divide the nonzero part of this range into the following intervals:

$$\{1\}, \{2\}, \{3, 4\}, \{5, 6, \ldots, 16\}, \{17, 18, \ldots, 2^{16} = 65536\},$$
$$\{65537, 65538, ..., 2^{65536}\}, \ldots$$

Each group is of the form $\{k + 1, k + 2, \ldots, 2^k\}$ where k is a power of 2.

The number of groups is $\log^* n$, which is defined to be the number of successive $\log$ operations that need to be applied to n to bring it down to 1 (or below 1).

If there are n elements, their rank values can range from 0 to $\log n$.

Divide the nonzero part of this range into the following intervals:

$$\{1\}, \{2\}, \{3, 4\}, \{5, 6, \ldots, 16\}, \{17, 18, \ldots, 2^{16} = 65536\},$$
$$\{65537, 65538, ..., 2^{65536}\}, \ldots$$

Each group is of the form $\{k + 1, k + 2, \ldots, 2^k\}$ where k is a power of 2.

The number of groups is $\log^* n$, which is defined to be the number of successive $\log$ operations that need to be applied to n to bring it down to 1 (or below 1).

- For instance, $\log^* 1000 = 4$ since $\log \log \log \log 1000 \leq 1$.

If there are n elements, their rank values can range from 0 to $\log n$.

Divide the nonzero part of this range into the following intervals:

$$\{1\}, \{2\}, \{3, 4\}, \{5, 6, \ldots, 16\}, \{17, 18, \ldots, 2^{16} = 65536\},$$
$$\{65537, 65538, ..., 2^{65536}\}, \ldots$$

Each group is of the form $\{k + 1, k + 2, \ldots, 2^k\}$ where k is a power of 2.

The number of groups is $\log^* n$, which is defined to be the number of successive $\log$ operations that need to be applied to n to bring it down to 1 (or below 1).

- For instance, $\log^* 1000 = 4$ since $\log \log \log \log 1000 \leq 1$.
- In practice there will just be the first five of the intervals shown; more are needed only if $n > 2^{65536}$, in other words never.

In a sequence of `find` operations, some may take longer than others.

Time Analysis

In a sequence of `find` operations, some may take longer than others.

We'll bound the overall running time using some creative accounting.

Time Analysis

In a sequence of `find` operations, some may take longer than others.

We'll bound the overall running time using some creative accounting.

We will give each node a certain amount of pocket money, such that the total money doled out is at most $n \log^* n$ dollars.

Time Analysis

In a sequence of `find` operations, some may take longer than others.

We'll bound the overall running time using some creative accounting.

We will give each node a certain amount of pocket money, such that the total money doled out is at most $n \log^* n$ dollars.

We will then show that each `find` takes $O(\log^* n)$ steps, plus some additional amount of time that can be paid for using the pocket money of the nodes involved - one dollar per unit of time.

Time Analysis

In a sequence of `find` operations, some may take longer than others.

We'll bound the overall running time using some creative accounting.

We will give each node a certain amount of pocket money, such that the total money doled out is at most $n \log^* n$ dollars.

We will then show that each `find` takes $O(\log^* n)$ steps, plus some additional amount of time that can be paid for using the pocket money of the nodes involved - one dollar per unit of time.

Thus the overall time for m `find`'s is $O(m \log^* n)$ plus at most $O(n \log^* n)$.

A node receives its allowance as soon as it ceases to be a root, at which point its rank is fixed.

A node receives its allowance as soon as it ceases to be a root, at which point its rank is fixed.

If this rank lies in the interval $\{k+1, \ldots, 2^k\}$, the node receives 2^k dollars.

A node receives its allowance as soon as it ceases to be a root, at which point its rank is fixed.

If this rank lies in the interval $\{k+1, \ldots, 2^k\}$, the node receives 2^k dollars.

By Lemma 3, the number of nodes with $\texttt{rank} > k$ is bounded by

$$\frac{n}{2^{k+1}} + \frac{n}{2^{k+2}} + \ldots \leq \frac{n}{2^k}$$

A node receives its allowance as soon as it ceases to be a root, at which point its rank is fixed.

If this rank lies in the interval $\{k+1, \ldots, 2^k\}$, the node receives 2^k dollars.

By Lemma 3, the number of nodes with $\texttt{rank} > k$ is bounded by

$$\frac{n}{2^{k+1}} + \frac{n}{2^{k+2}} + \ldots \leq \frac{n}{2^k}$$

Therefore the total money given to nodes in this particular interval is at most n dollars, and since there are $\log^* n$ intervals, the total money disbursed to all nodes is $n \log^* n$.

Time Analysis

Now, the time taken by a specific `find` is simply the number of pointers followed.

Time Analysis

Now, the time taken by a specific `find` is simply the number of pointers followed.

Consider the ascending rank values along this chain of nodes up to the root.

Now, the time taken by a specific `find` is simply the number of pointers followed.

Consider the ascending rank values along this chain of nodes up to the root.

Nodes x on the chain fall into two categories:

Now, the time taken by a specific `find` is simply the number of pointers followed.

Consider the ascending rank values along this chain of nodes up to the root.

Nodes x on the chain fall into two categories:

- either the rank of $\pi(x)$ is in a higher interval than the rank of x,

Now, the time taken by a specific `find` is simply the number of pointers followed.

Consider the ascending rank values along this chain of nodes up to the root.

Nodes x on the chain fall into two categories:

- either the rank of $\pi(x)$ is in a higher interval than the rank of x,
- or else it lies in the same interval.

Now, the time taken by a specific `find` is simply the number of pointers followed.

Consider the ascending rank values along this chain of nodes up to the root.

Nodes x on the chain fall into two categories:

- either the rank of $\pi(x)$ is in a higher interval than the rank of x,
- or else it lies in the same interval.

There are at most $\log^* n$ nodes of the first type, so the work done on them takes $O(\log^* n)$ time.

Time Analysis

Now, the time taken by a specific `find` is simply the number of pointers followed.

Consider the ascending rank values along this chain of nodes up to the root.

Nodes x on the chain fall into two categories:

- either the rank of $\pi(x)$ is in a higher interval than the rank of x,
- or else it lies in the same interval.

There are at most $\log^* n$ nodes of the first type, so the work done on them takes $O(\log^* n)$ time.

The remaining nodes - whose parents' ranks are in the same interval as theirs - have to pay a dollar out of their pocket money for their processing time.

This only works if the initial allowance of each node x is enough to cover all of its payments in the sequence of find operations.

This only works if the initial allowance of each node x is enough to cover all of its payments in the sequence of find operations.

Here's the crucial observation: each time x pays a dollar, its parent changes to one of higher rank.

This only works if the initial allowance of each node x is enough to cover all of its payments in the sequence of find operations.

Here's the crucial observation: each time x pays a dollar, its parent changes to one of higher rank.

Therefore, if x's rank lies in the interval $\{k+1, \ldots, 2^k\}$, it has to pay at most 2^k dollars before its parent's rank is in a higher interval; whereupon it never has to pay again.